In [1]:
```
import numpy as np
import pandas as pd
```

In [7]:
```
data=pd.read_excel(r'C:\Users\honey\Downloads\Rawdata.xlsx')
```

In [8]:
```
data
```

Out[8]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [10]:
```
id(data)
```

Out[10]:
```
2742419436080
```

In [11]:
```
data.columns
```

Out[11]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [12]:
```
data.shape
```

Out[12]:
```
(6, 6)
```

In [18]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       4 non-null      object
 3   Location  4 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

In [19]:
```
data.isnull()
```

Out[19]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [20]:
```
data.isna()
```

Out[20]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [22]:
```
data.isnull().sum()
```

Out[22]:
```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
dtype: int64
```

In [23]:
```
data.dtypes
```

Out[23]:
```
Name        object
Domain      object
Age         object
Location    object
Salary      object
Exp         object
dtype: object
```

In [24]:
```
type(data)
```

Out[24]:
```
pandas.core.frame.DataFrame
```

In [25]:
```
data.describe()
```

Out[25]:

	Name	Domain	Age	Location	Salary	Exp
count	6	6	4	4	6	5
unique	6	6	4	4	6	5
top	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
freq	1	1	1	1	1	1

In [26]:
```
data['Name']
```

Out[26]:
```
0      Mike
1    Teddy^
2     Uma#r
3      Jane
4    Uttam*
5       Kim
Name: Name, dtype: object
```

In [27]:
```
data['Name']=data['Name'].str.replace('\W','',regex=True)
```

In [28]:
```
data['Name']
```

Out[28]:
```
0     Mike
1    Teddy
2     Umar
3     Jane
4    Uttam
5      Kim
Name: Name, dtype: object
```

In [29]:
```
data
```

Out[29]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy	Testing	45' yr	Bangalore	10%%000	<3
2	Umar	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [31]:
```
data['Domain']=data['Domain'].str.replace('\W','',regex=True)
```

In [32]:
```
data
```

Out[32]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34 years	Mumbai	5^00#0	2+
1	Teddy	Testing	45' yr	Bangalore	10%%000	<3
2	Umar	Dataanalyst	NaN	NaN	1$5%000	4> yrs
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [33]:
```
data['Age']=data['Age'].str.extract('(\d+)')
data
```

Out[33]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5^00#0	2+
1	Teddy	Testing	45	Bangalore	10%%000	<3
2	Umar	Dataanalyst	NaN	NaN	1$5%000	4> yrs
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN
4	Uttam	Statistics	67	NaN	30000-	5+ year
5	Kim	NLP	55	Delhi	6000^$0	10+

In [34]:
```
data['Lcation']=data['Location'].str.replace(r'\W','',regex=True)
```

In [35]:
```
data
```

Out[35]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34	Mumbai	5^00#0	2+	Mumbai
1	Teddy	Testing	45	Bangalore	10%%000	<3	Bangalore
2	Umar	Dataanalyst	NaN	NaN	1$5%000	4> yrs	NaN
3	Jane	Analytics	NaN	Hyderbad	2000^0	NaN	Hyderbad
4	Uttam	Statistics	67	NaN	30000-	5+ year	NaN
5	Kim	NLP	55	Delhi	6000^$0	10+	Delhi

In [36]:
```
data['Salary']=data['Salary'].str.replace(r'\W','',regex=True)
```

In [37]:
```
data
```

Out[37]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34	Mumbai	5000	2+	Mumbai
1	Teddy	Testing	45	Bangalore	10000	<3	Bangalore
2	Umar	Dataanalyst	NaN	NaN	15000	4> yrs	NaN
3	Jane	Analytics	NaN	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67	NaN	30000	5+ year	NaN
5	Kim	NLP	55	Delhi	60000	10+	Delhi

In [39]:
```
data['Exp']=data['Exp'].str.extract('(\d+)')
```

In [40]:
```
data
```

Out[40]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	NaN	NaN	15000	4	NaN
3	Jane	Analytics	NaN	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67	NaN	30000	5	NaN
5	Kim	NLP	55	Delhi	60000	10	Delhi

In [42]:
```
clean_data=data.copy()
clean_data
```

Out[42]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	NaN	NaN	15000	4	NaN
3	Jane	Analytics	NaN	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67	NaN	30000	5	NaN
5	Kim	NLP	55	Delhi	60000	10	Delhi

In [44]:
```
clean_data.isnull().sum()
```

Out[44]:
```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
Lcation     2
dtype: int64
```

In [45]:
```
data['Age']
```

Out[45]:
```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

In [62]:
```
clean_data['Age'] = pd.to_numeric(
    clean_data['Age']).fillna(
    clean_data['Age'].mean()
)
```

In [63]:
```
clean_data
```

Out[63]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34.00	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45.00	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	50.25	NaN	15000	4	NaN
3	Jane	Analytics	50.25	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67.00	NaN	30000	5	NaN
5	Kim	NLP	55.00	Delhi	60000	10	Delhi

In [66]:
```
clean_data['Age']=pd.to_numeric(clean_data['Age']).fillna(clean_data['Age'].mean
clean_data['Age']
```

Out[66]:
```
0    34.00
1    45.00
2    50.25
3    50.25
4    67.00
5    55.00
Name: Age, dtype: float64
```

In [71]:
```
clean_data['Location']=clean_data['Location'].fillna(clean_data['Location'].mode
clean_data
```

Out[71]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34.00	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45.00	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	50.25	Hyderbad	15000	4	NaN
3	Jane	Analytics	50.25	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67.00	Hyderbad	30000	5	NaN
5	Kim	NLP	55.00	Delhi	60000	10	Delhi

In [69]:
```
data
```

Out[69]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	NaN	NaN	15000	4	NaN
3	Jane	Analytics	NaN	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67	NaN	30000	5	NaN
5	Kim	NLP	55	Delhi	60000	10	Delhi

In [72]:
```
data.mode()
```

Out[72]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Jane	Analytics	34	Bangalore	10000	10	Bangalore
1	Kim	Dataanalyst	45	Delhi	15000	2	Delhi
2	Mike	Datascience	55	Hyderbad	20000	3	Hyderbad
3	Teddy	NLP	67	Mumbai	30000	4	Mumbai
4	Umar	Statistics	NaN	NaN	5000	5	NaN
5	Uttam	Testing	NaN	NaN	60000	NaN	NaN

In [77]:
```
data['Location'].value_counts(dropna=False)
data['Location'].mode()
```

Out[77]:
```
0    Bangalore
1        Delhi
2     Hyderbad
3       Mumbai
Name: Location, dtype: object
```

In [79]:
```
data['Location'].mode()[0]
```

Out[79]:
```
'Bangalore'
```

In [80]:
```
id(clean_data)
```

Out[80]:
```
2742452773040
```

In [81]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       6 non-null      float64
 3   Location  6 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
 6   Lcation   4 non-null      object
dtypes: float64(1), object(6)
memory usage: 468.0+ bytes
```

In [82]:
```
clean_data
```

Out[82]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34.00	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45.00	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	50.25	Hyderbad	15000	4	NaN
3	Jane	Analytics	50.25	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67.00	Hyderbad	30000	5	NaN
5	Kim	NLP	55.00	Delhi	60000	10	Delhi

In [93]:
```
clean_data['Exp']=pd.to_numeric(clean_data['Exp']).fillna(clean_data['Exp'].mean
clean_data['Exp']
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[93], line 1
----> 1 clean_data['Exp']=pd.to_numeric(clean_data['Exp']).fillna(clean_data[
  ].mean())
      2 clean_data['Exp']

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:6570, in Series.mean(sel
f, axis, skipna, numeric_only, **kwargs)
   6562 @doc(make_doc("mean", ndim=1))
   6563 def mean(
   6564     self,
   (...)   6568     **kwargs,
   6569 ):
-> 6570     return NDFrame.mean(self, axis, skipna, numeric_only, **kwargs)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:12485, in NDFrame.mean
(self, axis, skipna, numeric_only, **kwargs)
  12478 def mean(
  12479     self,
  12480     axis: Axis | None = 0,
   (...)  12483     **kwargs,
  12484 ) -> Series | float:
> 12485     return self._stat_function(
  12486                 , nanops.nanmean, axis, skipna, numeric_only, **kwargs
  12487     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:12442, in NDFrame._stat
_function(self, name, func, axis, skipna, numeric_only, **kwargs)
  12438 nv.validate_func(name, (), kwargs)
  12440 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 12442 return self._reduce(
  12443     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_only
  12444 )

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:6478, in Series._reduce
(self, op, name, axis, skipna, numeric_only, filter_type, **kwds)
   6473     # GH#47500 - change to TypeError to match other methods
   6474     raise TypeError(
   6475         f"Series.{name} does not allow {kwd_name}={numeric_only} "
   6476         "with non-numeric dtypes."
   6477     )
-> 6478 return op(delegate, skipna=skipna, **kwds)

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:147, in bottleneck_switc
h.__call__.<locals>.f(values, axis, skipna, **kwds)
    145         result = alt(values, axis=axis, skipna=skipna, **kwds)
    146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kwds)
    149 return result

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:404, in _datetimelike_co
mpat.<locals>.new_func(values, axis, skipna, mask, **kwargs)
    401 if datetimelike and mask is None:
    402     mask = isna(values)
--> 404 result = func(values, axis=axis, skipna=skipna, mask=mask, **kwargs)
    406 if datetimelike:
    407     result = _wrap_results(result, orig_values.dtype, fill_value=iNaT)

File ~\anaconda3\Lib\site-packages\pandas\core\nanops.py:719, in nanmean(values,
```

```
axis, skipna, mask)
    716     dtype_count = dtype
    718 count = _get_counts(values.shape, mask, axis, dtype=dtype_count)
--> 719 the_sum = values.sum(axis, dtype=dtype_sum)
    720 the_sum = _ensure_numeric(the_sum)
    722 if axis is not None and getattr(the_sum, "ndim", False):

File ~\AppData\Roaming\Python\Python313\site-packages\numpy\_core\_methods.py:51,
in _sum(a, axis, dtype, out, keepdims, initial, where)
     49 def _sum(a, axis=None, dtype=None, out=None, keepdims=False,
     50          initial=_NoValue, where=True):
---> 51     return umr_sum(a, axis, dtype, out, keepdims, initial, where)

TypeError: can only concatenate str (not "int") to str
```

In [85]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       6 non-null      float64
 3   Location  6 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
 6   Lcation   4 non-null      object
dtypes: float64(1), object(6)
memory usage: 468.0+ bytes
```

In [86]:
```
clean_data
```

Out[86]:

	Name	Domain	Age	Location	Salary	Exp	Lcation
0	Mike	Datascience	34.00	Mumbai	5000	2	Mumbai
1	Teddy	Testing	45.00	Bangalore	10000	3	Bangalore
2	Umar	Dataanalyst	50.25	Hyderbad	15000	4	NaN
3	Jane	Analytics	50.25	Hyderbad	20000	NaN	Hyderbad
4	Uttam	Statistics	67.00	Hyderbad	30000	5	NaN
5	Kim	NLP	55.00	Delhi	60000	10	Delhi

In [90]:
```
clean_data.drop('Lcation',inplace=True)
clean_data
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[90], line 1
----> 1 clean_data.drop(         ,inplace=True)
      2 clean_data

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:5603, in DataFrame.drop(s
elf, labels, axis, index, columns, level, inplace, errors)
   5455 def drop(
   5456     self,
   5457     labels: IndexLabel | None = None,
   (...)   5464     errors: IgnoreRaise = "raise",
   5465 ) -> DataFrame | None:
   5466     """
   5467     Drop specified labels from rows or columns.
   5468
   (...)   5601             weight  1.0     0.8
   5602     """
-> 5603     return super().drop(
   5604         labels=labels,
   5605         axis=axis,
   5606         index=index,
   5607         columns=columns,
   5608         level=level,
   5609         inplace=inplace,
   5610         errors=errors,
   5611     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4810, in NDFrame.drop(s
elf, labels, axis, index, columns, level, inplace, errors)
   4808 for axis, labels in axes.items():
   4809     if labels is not None:
-> 4810         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4812 if inplace:
   4813     self._update_inplace(obj)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4852, in NDFrame._drop_
axis(self, labels, axis, level, errors, only_slice)
   4850         new_axis = axis.drop(labels, level=level, errors=errors)
   4851     else:
-> 4852         new_axis = axis.drop(labels, errors=errors)
   4853     indexer = axis.get_indexer(new_axis)
   4855 # Case for non-unique axis
   4856 else:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7136, in Index.dro
p(self, labels, errors)
   7134 if mask.any():
   7135     if errors != "ignore":
-> 7136         raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7137     indexer = indexer[~mask]
   7138 return self.delete(indexer)

KeyError: "['Lcation'] not found in axis"
```

In [91]:

```
clean_data['Age'] = clean_data['Age'].astype(int)
clean_data['Exp'] = clean_data['Exp'].astype(int)
clean_data['Age'] = clean_data['Age'].astype(int)
clean_data['Name'] = clean_data['Name'].astype('category')
```

```
clean_data['Domain'] = clean_data['Domain'].astype('category')
clean_data['Location'] = clean_data['Location'].astype('category')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[91], line 2
      1 clean_data['Age'] = clean_data['Age'].astype(int)
----> 2 clean_data['Exp'] = clean_data[      ].astype(int)
      3 clean_data['Age'] = clean_data['Age'].astype(int)
      4 clean_data['Name'] = clean_data['Name'].astype('category')

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:6665, in NDFrame.astype
(self, dtype, copy, errors)
   6659     results = [
   6660         ser.astype(dtype, copy=copy, errors=errors) for _, ser in self.it
ems()
   6661     ]
   6663 else:
   6664     # else, only a single dtype is given
-> 6665     new_data = self._mgr.astype(dtype=dtype, copy=copy, errors=errors)
   6666     res = self._constructor_from_mgr(new_data, axes=new_data.axes)
   6667     return res.__finalize__(self, method="astype")

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:449, in Base
BlockManager.astype(self, dtype, copy, errors)
    446 elif using_copy_on_write():
    447     copy = False
--> 449 return self.apply(
    450             ,
    451     dtype=dtype,
    452     copy=copy,
    453     errors=errors,
    454     using_cow=using_copy_on_write(),
    455 )

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:363, in Base
BlockManager.apply(self, f, align_keys, **kwargs)
    361         applied = b.apply(f, **kwargs)
    362     else:
--> 363         applied = getattr(b, f)(**kwargs)
    364     result_blocks = extend_blocks(applied, result_blocks)
    366 out = type(self).from_blocks(result_blocks, self.axes)

File ~\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:784, in Block.
astype(self, dtype, copy, errors, using_cow, squeeze)
    781         raise ValueError("Can not squeeze with more than one column.")
    782     values = values[0, :]  # type: ignore[call-overload]
--> 784 new_values = astype_array_safe(values, dtype, copy=copy, errors=errors)
    786 new_values = maybe_coerce_values(new_values)
    788 refs = None

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:237, in astype_ar
ray_safe(values, dtype, copy, errors)
    234     dtype = dtype.numpy_dtype
    236 try:
--> 237     new_values = astype_array(values, dtype, copy=copy)
    238 except (ValueError, TypeError):
    239     # e.g. _astype_nansafe can fail on object-dtype of strings
    240     #  trying to convert to float
    241     if errors == "ignore":

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:182, in astype_ar
ray(values, dtype, copy)
```

```
    179     values = values.astype(dtype, copy=copy)
    181 else:
--> 182     values = _astype_nansafe(values, dtype, copy=copy)
    184 # in pandas we don't store numpy str dtypes, so convert to object
    185 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):

File ~\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:133, in _astype_n
ansafe(arr, dtype, copy, skipna)
    129     raise ValueError(msg)
    131 if copy or arr.dtype == object or dtype == object:
    132     # Explicit copy, or required since NumPy can't view from / to object.
--> 133     return arr.astype(dtype, copy=True)
    135 return arr.astype(dtype, copy=copy)

ValueError: cannot convert float NaN to integer
```

In [94]:
```
clean_data['Exp'].unique()
```

Out[94]:
```
array(['2', '3', '4', nan, '5', '10'], dtype=object)
```

In [103…
```
exp_num=pd.to_numeric(clean_data['Exp'])
clean_data['Exp'] = exp_num.fillna(exp_num.mean())
```

In [96]:
```
clean_data['Exp'].unique()
```

Out[96]:
```
array(['2', '3', '4', nan, '5', '10'], dtype=object)
```

In [99]:
```
clean_data
```

Out[99]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2.0
1	Teddy	Testing	45	Bangalore	10000	3.0
2	Umar	Dataanalyst	50	Hyderbad	15000	4.0
3	Jane	Analytics	50	Hyderbad	20000	4.8
4	Uttam	Statistics	67	Hyderbad	30000	5.0
5	Kim	NLP	55	Delhi	60000	10.0

In [102…
```
clean_data['Exp'].dtype
```

Out[102…
```
dtype('float64')
```

In [104…
```
clean_data
```

Out[104…

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2.0
1	Teddy	Testing	45	Bangalore	10000	3.0
2	Umar	Dataanalyst	50	Hyderbad	15000	4.0
3	Jane	Analytics	50	Hyderbad	20000	4.8
4	Uttam	Statistics	67	Hyderbad	30000	5.0
5	Kim	NLP	55	Delhi	60000	10.0

In [105…
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Name      6 non-null      object 
 1   Domain    6 non-null      object 
 2   Age       6 non-null      int64  
 3   Location  6 non-null      object 
 4   Salary    6 non-null      object 
 5   Exp       6 non-null      float64
dtypes: float64(1), int64(1), object(4)
memory usage: 420.0+ bytes
```

In [109…
```
clean_data['Age'] = clean_data['Age'].astype(int)
clean_data['Exp'] = clean_data['Exp'].astype(int)
clean_data['Salary']=clean_data['Salary'].astype(int)
clean_data['Name'] = clean_data['Name'].astype('category')
clean_data['Domain'] = clean_data['Domain'].astype('category')
clean_data['Location'] = clean_data['Location'].astype('category')
```

In [110…
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype   
---  ------    --------------  -----   
 0   Name      6 non-null      category
 1   Domain    6 non-null      category
 2   Age       6 non-null      int64   
 3   Location  6 non-null      category
 4   Salary    6 non-null      int64   
 5   Exp       6 non-null      int64   
dtypes: category(3), int64(3)
memory usage: 938.0 bytes
```

In [111…
```
clean_data
```

Out[111...

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	Hyderbad	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [112...
```
clean_data.to_csv('clean_data.csv')
```

In [118...
```
import os
print(os.getcwd)
```

```
<built-in function getcwd>
```

In [115...
```
df=pd.read_csv(r'C:\Users\honey\clean_data.csv')
```

In [116...
```
df
```

Out[116...

	Unnamed: 0	Name	Domain	Age	Location	Salary	Exp
0	0	Mike	Datascience	34	Mumbai	5000	2
1	1	Teddy	Testing	45	Bangalore	10000	3
2	2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	3	Jane	Analytics	50	Hyderbad	20000	4
4	4	Uttam	Statistics	67	Hyderbad	30000	5
5	5	Kim	NLP	55	Delhi	60000	10

In [117...
```
clean_data.to_csv(
    r'C:\Users\honey\Downloads\clean_data.csv',
    index=False
)
```

In [ ]: